DE MONTFORT UNIVERSITY CAMBODIA

Operating Systems & Networks
CTEC1004C

Lecture – 2 : Number Representation

Dr. Sameer Jha, SFHEA

Faculty of Computing

De Montfort University Cambodia

DE MONTFORT UNIVERSITY CAMBODIA

Index

- Number representation
- 2s-Complement Signed Integers
- Signed Negation
- Overflow
- Floating Point Standard

DE MONTFORT UNIVERSITY CAMBODIA

Introduction

- **Number System** is a method of representing numbers with the help of a set of symbols and rules; it is a mathematical notation used to represent quantities or values in various forms.
- *Please remember every piece of data in a computer—whether it's text, images, sound, video, code, or memory addresses—is ultimately stored as binary, a string of 0s and 1s making numbers and number systems the core of computing.*

DE MONTFORT UNIVERSITY CAMBODIA

Number representation

- So far we learnt how to represent unsigned decimal numbers in Binary.
- We can represent the decimal number -5 as binary -101
- But, on the hardware level, the computer only knows bits, it does not have the ‘minus’ sign!

DE MONTFORT UNIVERSITY CAMBODIA

2s-Complement Signed Integers

- Need to learn how to represent signed numbers.
- In n-bit number, the **left-most bit** indicates the sign
  - Left-most bit is 0: positive number
  - Left-most bit is 1: negative number
- Range: -2^{n-1} to $+2^{n-1} - 1$
  - Using 32 bits: –2,147,483,648 to +2,147,483,647
- Some specific numbers
  - 0: 0000 0000 … 0000
  - –1: 1111 1111 … 1111
  - Most-negative (smallest): 1000 0000 … 0000
  - Most-positive (largest): 0111 1111 … 1111

DE MONTFORT UNIVERSITY CAMBODIA

2s-Complement Signed Integers

- Non-negative numbers have the same unsigned and 2s-complement representation
- Representation for negative numbers:

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- For example:

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$

$= -1\times2^{31} + 1\times2^{30} + \ldots + 1\times2^2 + 0\times2^1 + 0\times2^0$

$= -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$

DE MONTFORT UNIVERSITY CAMBODIA

Signed Negation

- Two's complement has a shortcut that allows to negate numbers quickly
- Invert all bits and add 1
  - Inverting bits means 1 → 0, 0 → 1
- Example: negate +2

  +2 = 0000 0000 … 0010_2

  −2 = 1111 1111 … 1101_2 + 1

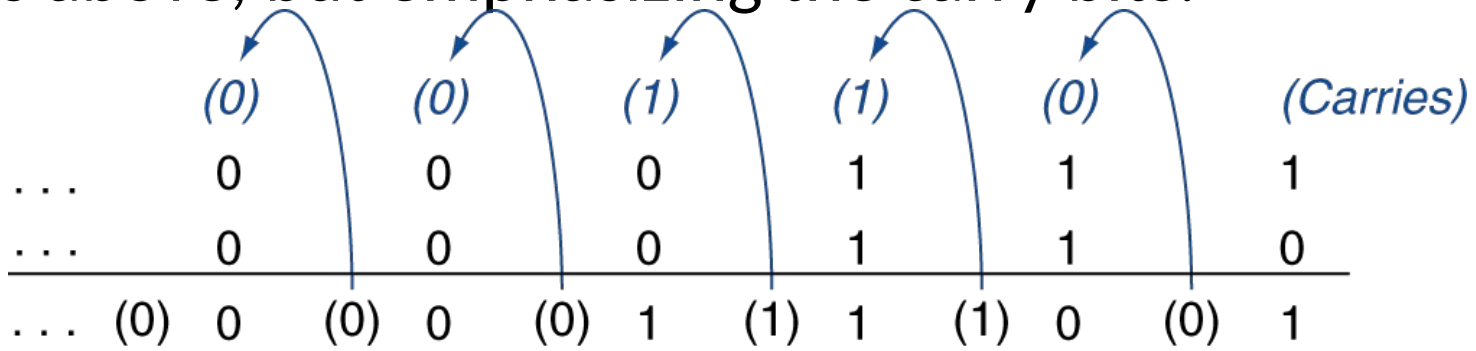
  = 1111 1111 … 1110_2

Integer Addition

- Example: 7 + 6

$$
\begin{array}{ll}
 & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{two} = 7_{ten} \\
+ & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0110_{two} = 6_{ten} \\
\hline
= & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1101_{two} = 13_{ten}
\end{array}
$$

- Same as above, but emphasizing the carry bits:



- Subtraction: same procedure: add negation of second operand (we already know how to negate in two's complement representation: flip bits and add 1)

DE MONTFORT UNIVERSITY CAMBODIA

Overflow

- Overflow: result of arithmetic operation is too big to be stored in the given number of bits
- Adding two large positive numbers -> result can overflow and be interpreted as a negative number!
- Detecting overflows:
  - No overflow possible if adding positive and negative number
  - Adding two positive operands: overflow if result sign is 1
  - Adding two negative operands: overflow if result sign is 0

DE MONTFORT UNIVERSITY CAMBODIA

Dealing with Overflow

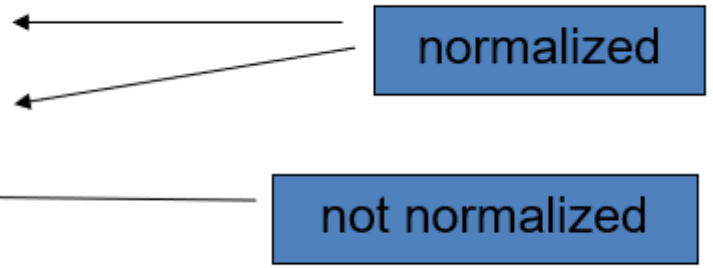
- Some languages (e.g., C) ignore overflow
  - Compilers will use MIPS *addu*, *addui*, *subu* instructions

- Other languages (e.g., Ada, Fortran) require raising an exception
  - Compilers use MIPS *add*, *addi*, *sub* instructions
  - On overflow, they invoke exception handler

DE MONTFORT UNIVERSITY CAMBODIA

Floating Point Numbers

- Representation for non-integral numbers
- Including very small and very large numbers
- Computer representation is similar to scientific notation:
  - $3.1 \times 10^{2} = 310$
  - $3.1 \times 10^{-2} = 0.031$
  - $+0.002 \times 10^{-4}$



- In binary: $\pm 1.011_2 \times 2^{101}$
- fraction exponent
- Types float and double in C

Floating Point Standard

- Defined by IEEE Standard 754-1985
- Two versions:
  - Single precision (32-bit)
  - Double precision (64-bit)
- Bit format:

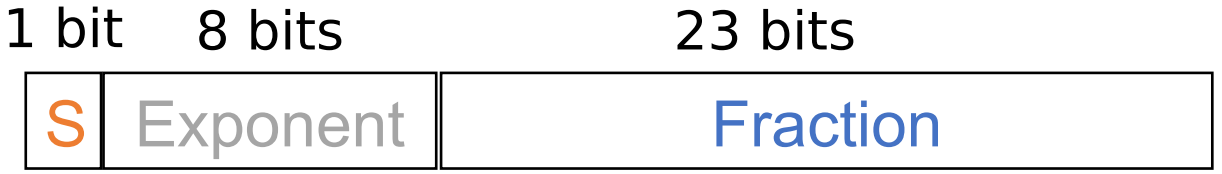
We will focus on single precision

- Interpretation / meaning:

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

DE MONTFORT UNIVERSITY CAMBODIA

IEEE Floating-Point Format



- S: **sign bit** (0 ⇒ non-negative, 1 ⇒ negative)

- **Significand:**
  - Digits left and right of radix point
  - Left: integer part, can be normalized so that it is always 1
    - ➢No need to represent it explicitly, called “hidden bit”
  - Right: 23 most significant digits are the **fraction**

- Exponent: 8-bit unsigned integer -> Range 0 to 255
  - Has to be shifted by a **bias** to get range from −128 to 127
  - Single-precision floating point: Bias = 127

DE MONTFORT UNIVERSITY CAMBODIA

Using float to represent integers?

- Only integers in [− 16 777 216 , 16 777 216] can be exactly represented
  - 16 777 216 = 2^{24}
  - 24 is the number of digits in the significand!
- Integers in [− 33 554 432 , − 16 777 217] or in [16 777 217 , 33 554 432] round to a multiple of 2
- Integers in [− 2^{26} , − 2^{25} − 1] or in [2^{25} + 1 , 2^{26}] round to a multiple of 4
- …
- Integers larger or equal than 2^{128} are rounded to "infinity".

DE MONTFORT UNIVERSITY CAMBODIA

Floating-Point Example

- What number is represented by this single-precision float?

  x = 11000000101000000000000000000000

  - Sign = 1
  - Fraction = $01000\ldots00_2$
  - Exponent = $10000001_2 = 129_{10}$

- $x = (-1)^1 \times (1.01_2) \times 2^{(129 - 127)}$

  $= (-1) \times 1.25_{10} \times 2^2$

  $= -5.0_{10}$

Step 1: extract sign bit, fraction, exponent

Step 2: write in scientific notation

Step 3: convert to decimal number

DE MONTFORT UNIVERSITY CAMBODIA

One bit string, many interpretations!

- A 32-bit string can mean very different things
- For example:
  - 1000 1100 0100 1011 0100 1000 0010 0000
  - (in hex: 0x8C4B4820)
  - Unsigned int: 2 353 743 904
  - Twos complement: -1 941 223 392
  - IEEE 754 Floating point: $-1.5660255 * 10^{-31}$
  - MIPS instruction: **LW $t3 0x4820 $v0**

DE MONTFORT UNIVERSITY CAMBODIA

Final thoughts about number representations

- Bits have no inherent meaning
  - Interpretation depends on the instructions applied
- Computer representations of numbers
  - Finite range and precision
  - **Need to account for this in programs**
- All instruction sets have support for arithmetic
  - Signed and unsigned integers
  - Floating-point numbers
- Bounded range and precision
  - Operations can overflow and underflow

DE MONTFORT UNIVERSITY CAMBODIA

DE MONTFORT UNIVERSITY CAMBODIA

DE MONTFORT UNIVERSITY CAMBODIA

Thank You!